Demo video script (3–5 minutes)

One live happy path, one injected failure plus one edge case, the guardrail code, and real metrics. Target length 4:30. Everything quoted below comes from the repository at `main` (`236e030`): numbers from `evals/results/` and the live runs of 2026-10-28, code lines from `src/predixion/`. The recording has not been made yet; once it is uploaded, add the link to submission.md.

Before recording

```
uv sync
cd frontend && npm ci && npm run build && cd ..
```

In `.env`: - `OPENAI_API_KEY`, `TAVILY_API_KEY` (and optionally `GROQ_API_KEY`). - `PREDIXION_MODE=live`, `SERVE_FRONTEND_DIR=frontend/dist`, `ENABLE_FAULT_INJECTION=true`. The last one enables the "simulate a failure" control; leave it off anywhere except a local demo.

Then: - `uv run predixion serve` and open http://127.0.0.1:8000. - Open these in editor tabs, scrolled to the line: `src/predixion/reliability.py:174` (budget reservation), `src/predixion/reliability.py:243` (the single retry owner), `src/predixion/grounding.py:90` (quote check), `src/predixion/grounding.py:524` (reference check), `src/predixion/urls.py:123` (URL validation), `src/predixion/prompts.py:387` (untrusted-text fencing). - Do one warm-up live run so the timing is familiar. Live runs took 28–90 s in the recorded evaluations; the question below took 28 s and $0.05 when it was run for the held-out set, but live results vary, so the script says what to narrate whichever way the run ends. - Close other tabs, browser zoom about 110%.

If live keys are unavailable, every step except the timing works in fixture mode. Say on camera that it is fixture mode; the UI labels it four ways.

Script

0:00–0:25 · Intro

On the landing page.

> "This is Predixion AI, a bounded research agent built for the AI startup intern assignment. It takes a research question, plans the criteria an answer needs, searches the web, reads the most useful pages, extracts quoted evidence, and writes a report in which every claim cites a passage you can open. The design rule throughout is: the model proposes, the code decides. In the next four minutes I'll show a live run, a failure, an edge case, the guardrail code, and the measured numbers."

0:25–0:55 · Architecture in one picture

Scroll the README to the Mermaid diagram.

> "One workflow, no swarm: plan, then up to two rounds of search, filter, read, extract, and assess coverage, then synthesize, verify every claim against its own quotes, repair once, finalize. Two tools: Tavily Search and Tavily Extract. One model behind strict JSON schemas, with Groq-hosted models as a cheaper first choice for the small planning steps. Every stop condition lives in code: rounds, pages, time, tokens, cost, and cancellation."

0:55–2:25 · Happy path (live)

Click **Start research**. Type: **Is Redis still open source?** Confirm **Live web research** is selected. Start.

While the plan appears:

> "The plan is the model's proposal: three criteria, each with a freshness requirement, because licensing changed over time. The code validates the plan, assigns the IDs, and decides how many searches and pages it gets. There is no fake progress bar; each stage is a server event."

When coverage appears:

> "Coverage is computed by code, per criterion. If a criterion were missing it would say which entity it is missing for, and the follow-up round would search for exactly that."

When the report appears (about 30–60 seconds):

- Read the short answer. If it says the answer depends on the version: "That's the honest shape of this question: BSD-3 up to 7.2, a choice of RSALv2, SSPLv1 or AGPLv3 from Redis 8."
- Scroll to **Sources disagree**: "Two sources disagreed; the report quotes both sides with dates and says which it relied on and why. Conflicts the code detects on its own are labelled 'possible conflict, needs review' instead of being resolved by guesswork."
- Click a citation such as **E3**: "The drawer shows the exact passage, the site, its date, the caveats, and, at the bottom, which claims depend only on this source. That last part is a dependency analysis, not a new fact check, and it says so." Press Escape; focus returns to the citation.
- Point at **Confidence**: "Confidence is a rule outcome computed from the evidence the surviving claims cite, not the model's self-rating. It lists its reasons."
- Open **Execution details**: "Real metrics for this run: duration, searches, pages read, model calls, tokens, and the estimated cost, a few cents. Stage timings show where the time goes: model calls, not retrieval."

- Click **Export Markdown**.

If the run ends `partial` instead of `completed`, say: "It stopped at the round limit with some criteria unresolved, and it says so in Limitations and Next steps rather than filling the gap."

2:25–3:20 · Failure case and edge case

Click **New research**. Type any question. Open **Advanced: simulate a failure (testing)**, choose **Search provider is down**. Start.

> "Search returns a 503 every time. The retry owner tries a bounded number of times with backoff, then stops. The run ends Failed with the reason, no findings, Low confidence, and a Retry button. It does not invent an answer, and a provider outage is never reported as 'no evidence exists'; those are different statuses in the code."

Then the edge case. New research: **What does QuantumLeap Vector Engine 9 from Hyperion DB cost?** (no fault; takes about 15–30 s live).

> "A product that does not exist. The agent searched, read pages, extracted nothing usable, and ended with Not enough evidence. The sample in the repository for this question read eight pages and invented no price."

3:20–4:05 · Guardrails in code

Switch to the editor, one tab per point, a few seconds each.

- `reliability.py:174`, `Budget.reserve_model_attempt`: "Before any model call, the worst-case tokens and cost are reserved against the run's ceiling. After the call the real usage is settled; if usage is unknown, the reservation stays spent. Retries, model fallbacks, and follow-up rounds all draw from the same budget."
- `reliability.py:243`, `call_with_retries`: "One retry owner. SDK retries are off (`max_retries=0` in `llm.py`), only transient kinds are retried, Retry-After is honored and capped, and a retry never sleeps past the deadline."
- `grounding.py:90` and `:524`: "A quote must appear verbatim in the passage after conservative normalization, or the evidence is rejected. A claim may cite only evidence IDs that exist in this run; unknown IDs are dropped and the claim is marked unsupported. Then a verifier judges each claim against only its own quotes, and unsupported claims are repaired once or removed."
- `urls.py:123` and `prompts.py:387`: "Every URL is validated before it is read or rendered: only http and https, no credentials, no private, loopback, link-local or metadata addresses, IPv6 included. Page text goes to the model inside fenced untrusted elements with the delimiters

escaped, and the model has no tools, so page text cannot change a budget, a destination, or an instruction."

4:05–4:30 · Evaluation numbers and close

Show `docs/evaluation.md` or `evals/results/live-gpt-6.1-terra.md`.

> "Evaluation: eight cases including the four assignment questions, a nonexistent product, an ambiguous question, contradictory sources, and a provider outage. Offline, the fixture suite passes 20 of 20 checks in about a second and runs in CI. Live, against a one-pass baseline on the same model, the agent covered about twice as many required criteria, 12 versus 6 of 30, at about 8 times the latency and 2.3 times the cost. Those are single runs, scored by a lexical proxy, and the document says so."

Show the terminal: `uv run pytest` (256 tests, offline) and the green CI run.

> "What's not done: runs aren't durable across restarts, there are no per-user quotas, and verification checks claims against quotes, not quotes against the world. All of that is written down in the README and the audit. The model proposes; the code decides. Thanks."

Things to avoid saying

- That the answers are guaranteed correct. Verification checks claims against their quotes, not the quotes against reality.
- That this is production-ready or deployed. It runs locally; the README lists what remains before public use.
- That the owner manually reviewed the evaluation, unless they have done so before recording.
- Any precise latency or cost figure other than the ones on screen or in `evals/results/`.